main

May 16, 2026

1 EM Fixed Income Analytics Suite

Programming choices: Start date = 2015-01-01, capturing multiple EM stress situations

List of problems faced:

- Data Sourcing: WRDS didn't work, despite successfully conecting my account it does not have the necessary info, had to manually download
- File names in Investing.com sometimes had typos -> quick fix by renaming, this even bypassed the column check
- 2Y Colombia resulted in 65.35% data loss, 20Y Hungary resulted in 43.42% data loss, both were removed. Next largest offender was 1Y romania but it wasn't due to a lack of reporting before a certain period (patchy), so it was kept

Things I want to better understand before turning the project in: - PCA analysis, finish watching 3Blue1Brown videos to better understand the process despite having the basics nailed down - Why was 1Y just noise for Romania

[12]:
```
import pandas as pd
import numpy as np
import glob
import re
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
from pathlib import Path
from collections import defaultdict
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
import missingno as msno
from matplotlib_inline import config
```

[13]:
```
plt.rcParams.update({
    "figure.figsize": (14, 5),
    "axes.grid": True,
    "grid.alpha": 0.3,
    "font.size": 11,
})

MACRO_EVENTS = {
```

```
    "2015-08-24": "CNY devaluation",
    "2016-11-09": "Trump election",
    "2018-05-01": "EM sell-off (Turkey/Argentina)",
    "2020-03-16": "Covid crash",
    "2021-03-17": "Fed dots hawkish shift",
    "2022-03-16": "Fed first hike (post-Covid)",
    "2022-09-26": "UK gilt crisis / EM contagion",
    "2023-10-19": "US 10Y hits 5%",
    "2024-09-18": "Fed pivot (first cut)",
    "2025-01-20": "Trump 2.0 inauguration",
}
```

1.1 1. Yield Curve PCA & Regime Detection

1.1.1 Downloading Data

Country Selection

[14]:
```
local_list_of_countries = ['Brazil','Mexico','South Africa','Poland']
hard_list_of_countries = ['Colombia', 'Hungary','Romania']
all_countries = sorted(set(local_list_of_countries + hard_list_of_countries))
```

Data Scraping Getting all the data for countries via Investing.com Maturities vary per country: maximizing coverage, avoiding redudancy between adjacent points when possible

As I don't have access to my Bloomberg while doing this project, I'll need to find another way to obtain Hard currency bond yields. Proxy spreads: local-currency yields - US treasury yields

Source: https://www.investing.com/rates-bonds/

Jordan was removed from the hard currency list due to limited data availability from Investing.com

Cleaning the Data

[15]:
```
base_dir = Path(r'data\raw')
all_unique_columns = set()
column_provenance = defaultdict(list)

for file_path in base_dir.rglob("*.csv"):
    try:
        raw_columns = pd.read_csv(file_path, nrows=0).columns.tolist()
        all_unique_columns.update(raw_columns)

        for col in raw_columns:
            column_provenance[col].append(file_path.name)
    except Exception as e:
        print(f"Failed to read header of {file_path.name}: {e}")

print(f"Total Unique Raw Columns Discovered: {len(all_unique_columns)}\n")
for col in sorted(all_unique_columns):
    file_count = len(column_provenance[col])
```

```
    print(f"'{col}' -> Found in {file_count} files")
```

```
Total Unique Raw Columns Discovered: 6

'Change %' -> Found in 32 files
'Date' -> Found in 32 files
'High' -> Found in 32 files
'Low' -> Found in 32 files
'Open' -> Found in 32 files
'Price' -> Found in 32 files
```

[16]:
```
files = glob.glob("data/raw/*/*.csv")
print("Files found:", len(files))

country_files = defaultdict(list)
for f in files:
    match = re.search(r"(\w[\w\s]+?)\s+(\d+)-Year Bond", f)
    if match:
        country  = match.group(1)
        maturity = match.group(2) + "Y"
        country_files[country].append((maturity, f))
    else:
        print(f"NO MATCH: {f}")

country_dfs = {}
for country, maturity_files in country_files.items():
    dfs = []
    for maturity, fname in maturity_files:
        df = pd.read_csv(fname, usecols=["Date", "Price"])
        df["Date"] = pd.to_datetime(df["Date"])
        df["Price"] = pd.to_numeric(df["Price"], errors="coerce")
        df = df.rename(columns={"Price": maturity})
        df = df.set_index("Date")
        dfs.append(df)

    combined = pd.concat(dfs, axis=1).sort_index().loc["2015-01-01":
 ↪"2026-05-01"]
    combined = combined[sorted(combined.columns, key=lambda x: int(x[:-1]))]
    country_dfs[country] = combined

for country, df in country_dfs.items():
    print(f"{country}: {df.shape} | columns: {list(df.columns)}")
```

```
Files found: 32
Brazil: (3016, 5) | columns: ['2Y', '3Y', '5Y', '8Y', '10Y']
Colombia: (3403, 4) | columns: ['4Y', '5Y', '10Y', '15Y']
Hungary: (2962, 4) | columns: ['1Y', '5Y', '10Y', '15Y']
Mexico: (3260, 5) | columns: ['3Y', '5Y', '10Y', '20Y', '30Y']
```

```
Poland: (3457, 5) | columns: ['1Y', '2Y', '3Y', '5Y', '10Y']
Romania: (3446, 4) | columns: ['3Y', '5Y', '7Y', '10Y']
South Africa: (2891, 5) | columns: ['5Y', '10Y', '12Y', '20Y', '30Y']
```

[17]:
```
for country, df in country_dfs.items():
    df.to_csv(f"data/output/{country}.csv")
    print(f"Saved {country}")
```

```
Saved Brazil
Saved Colombia
Saved Hungary
Saved Mexico
Saved Poland
Saved Romania
Saved South Africa
```

1.1.2 PCA

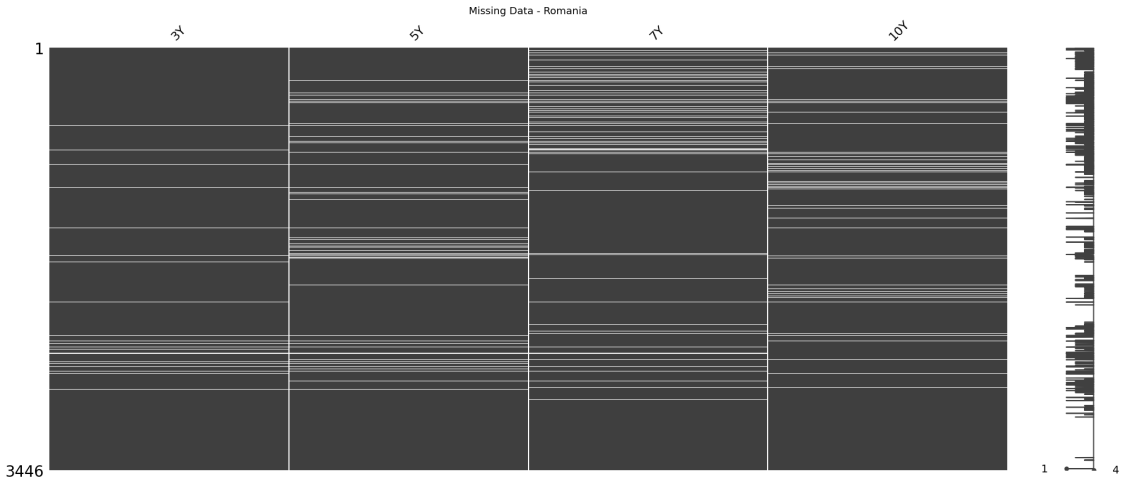
[18]:
```
change_dfs = {}

for country, df in country_dfs.items():
    dy = df.diff().dropna()
    dy = dy.dropna()

    change_dfs[country] = dy
    pct_lost = 100 * (1 - len(dy) / len(df))
    print(f"For {country}: {len(dy)} observations left after calculating␣
 ↪difference and dropping N/As ({pct_lost:.2f}% of data lost)")
```

```
For Brazil: 2698 observations left after calculating difference and dropping
N/As (10.54% of data lost)
For Colombia: 2847 observations left after calculating difference and dropping
N/As (16.34% of data lost)
For Hungary: 2748 observations left after calculating difference and dropping
N/As (7.22% of data lost)
For Mexico: 2778 observations left after calculating difference and dropping
N/As (14.79% of data lost)
For Poland: 2640 observations left after calculating difference and dropping
N/As (23.63% of data lost)
For Romania: 2685 observations left after calculating difference and dropping
N/As (22.08% of data lost)
For South Africa: 2089 observations left after calculating difference and
dropping N/As (27.74% of data lost)
```

Inspecting missing data to make sure PCA runs properly: removed 2Y Colombia, 20Y Hungary

[19]:
```
c = "Romania"
df = country_dfs[c]
msno.matrix(df)
```

```
plt.title(f"Missing Data - {c}")
plt.show()
```



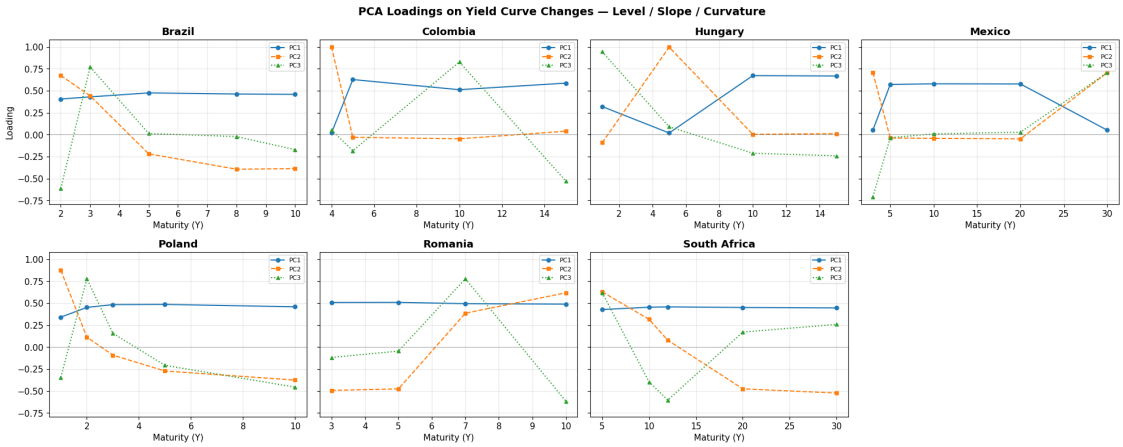
[20]:
```
pca_results = {}
for country, dy in change_dfs.items():
    scaler = StandardScaler()
    dy_std = scaler.fit_transform(dy)

    pca = PCA(n_components=3)
    scores = pca.fit_transform(dy_std)

    scores_df = pd.DataFrame(
        scores,
        index=dy.index,
        columns=["PC1 (level)", "PC2 (slope)", "PC3 (curvature)"],
    )
    loadings_df = pd.DataFrame(
        pca.components_.T,
        index=dy.columns,
        columns=["PC1", "PC2", "PC3"],
    )

    pca_results[country] = {
        "pca": pca,
        "scaler": scaler,
        "scores": scores_df,
        "loadings": loadings_df,
    }

    evr = pca.explained_variance_ratio_
```

```
    print(
        f"{country}: PC1={evr[0]:.1%}  PC2={evr[1]:.1%}  PC3={evr[2]:.1%}  "
        f"cumulative={evr.sum():.1%}"
    )
```

```
Brazil: PC1=76.3%  PC2=11.4%  PC3=6.1%  cumulative=93.7%
Colombia: PC1=53.1%  PC2=25.0%  PC3=15.7%  cumulative=93.8%
Hungary: PC1=51.8%  PC2=25.0%  PC3=21.9%  cumulative=98.8%
Mexico: PC1=56.4%  PC2=36.2%  PC3=3.7%  cumulative=96.3%
Poland: PC1=72.6%  PC2=14.0%  PC3=7.1%  cumulative=93.8%
Romania: PC1=68.4%  PC2=12.0%  PC3=10.8%  cumulative=91.1%
South Africa: PC1=89.9%  PC2=6.9%  PC3=2.1%  cumulative=99.0%
```

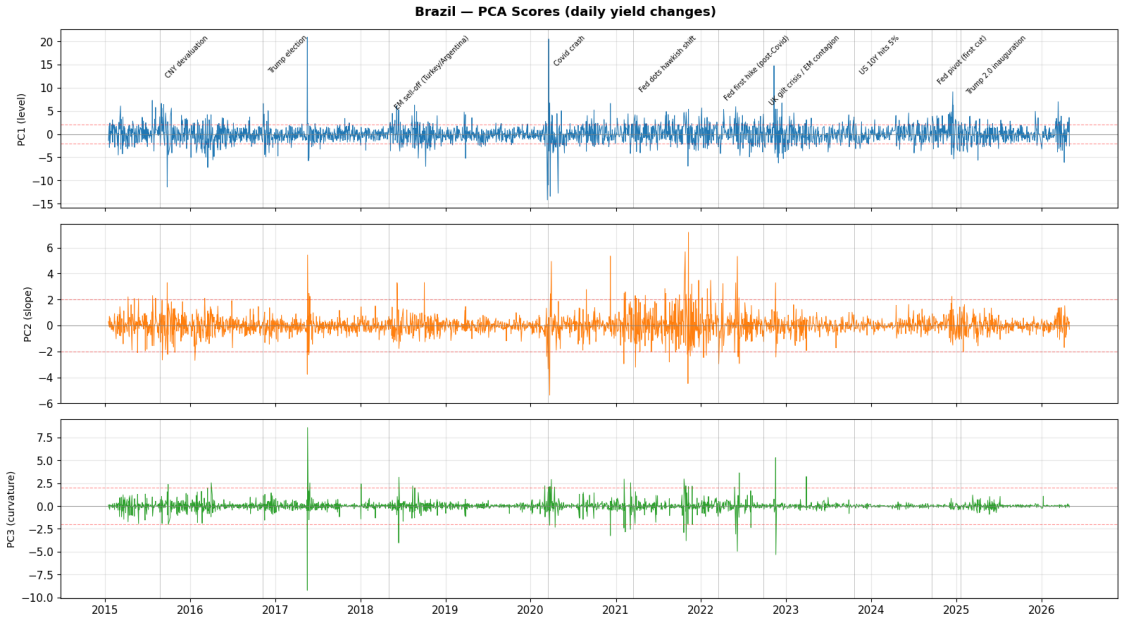
Removed Romania 1Y as it was introducing noise

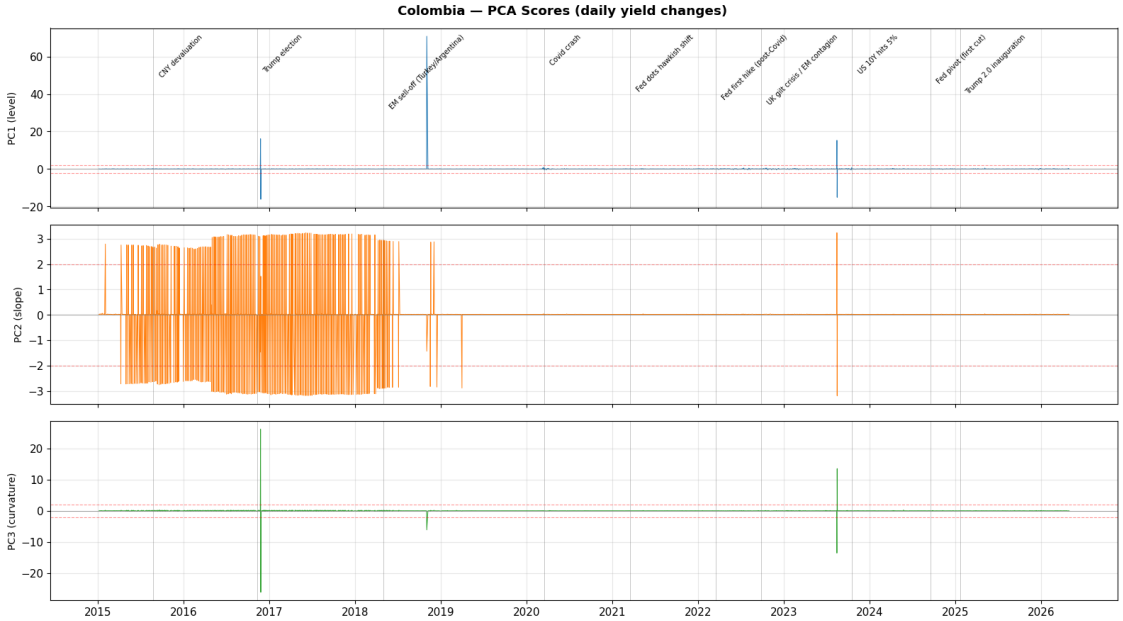
[21]:
```
fig, axes = plt.subplots(2, 4, figsize=(20, 8), sharey=True)
axes = axes.flatten()

for i, (country, res) in enumerate(sorted(pca_results.items())):
    ax = axes[i]
    loadings = res["loadings"]
    maturities = [int(m[:-1]) for m in loadings.index]

    for pc_col, style in zip(loadings.columns, ["-o", "--s", ":^"]):
        ax.plot(maturities, loadings[pc_col], style, label=pc_col, markersize=5)

    ax.set_title(country, fontweight="bold")
    ax.set_xlabel("Maturity (Y)")
    ax.axhline(0, color="grey", linewidth=0.5)
    if i == 0:
        ax.set_ylabel("Loading")
    ax.legend(fontsize=8)

# Hide the 8th (empty) subplot
axes[-1].set_visible(False)
fig.suptitle(
    "PCA Loadings on Yield Curve Changes - Level / Slope / Curvature",
    fontsize=14,
    fontweight="bold",
)
fig.tight_layout()
plt.savefig("data/output/pca_loadings.png", dpi=150, bbox_inches="tight")
plt.show()
```

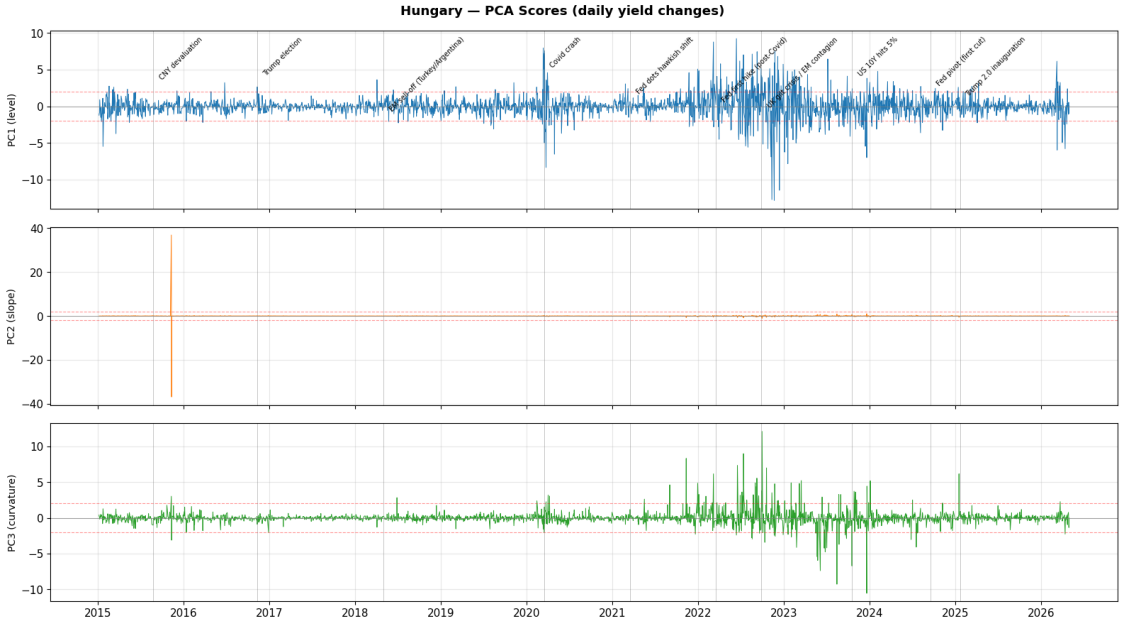


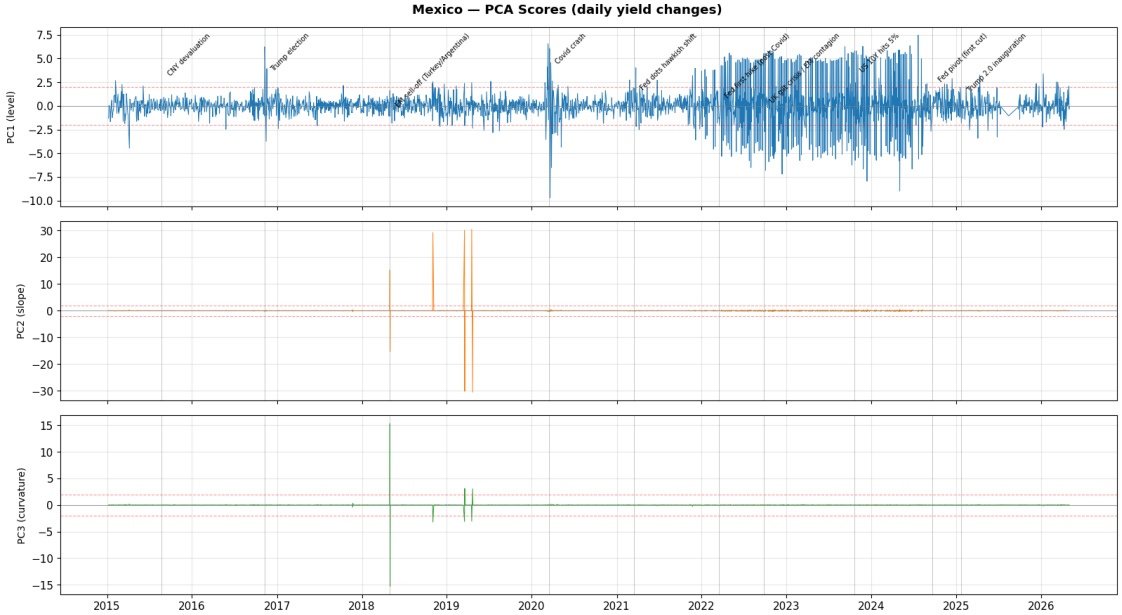
[22]:
```python
def plot_pc_scores(country, scores_df, macro_events):
    """Plot 3 PC score time series with macro event annotations."""
    fig, axes = plt.subplots(3, 1, figsize=(16, 9), sharex=True)

    colors = ["#1f77b4", "#ff7f0e", "#2ca02c"]
    for j, col in enumerate(scores_df.columns):
        ax = axes[j]
        ax.plot(scores_df.index, scores_df[col], linewidth=0.7, color=colors[j])
        ax.set_ylabel(col, fontsize=10)
        ax.axhline(0, color="grey", linewidth=0.5)

        # ±2 bands (z-score alert threshold from blueprint §1.4)
        ax.axhline(2, color="red", linestyle="--", alpha=0.4, linewidth=0.8)
        ax.axhline(-2, color="red", linestyle="--", alpha=0.4, linewidth=0.8)

        # Macro event annotations
        for date_str, label in macro_events.items():
            evt_date = pd.Timestamp(date_str)
            if evt_date in scores_df.index or (
                scores_df.index.min() <= evt_date <= scores_df.index.max()
            ):
                ax.axvline(evt_date, color="grey", alpha=0.5, linewidth=0.6)
                if j == 0:  # annotate only on the top panel
                    ax.annotate(
                        label,
                        xy=(evt_date, ax.get_ylim()[1]),
                        xytext=(5, -5),
                        textcoords="offset points",
                        fontsize=7,
                        rotation=45,
                        va="top",
```

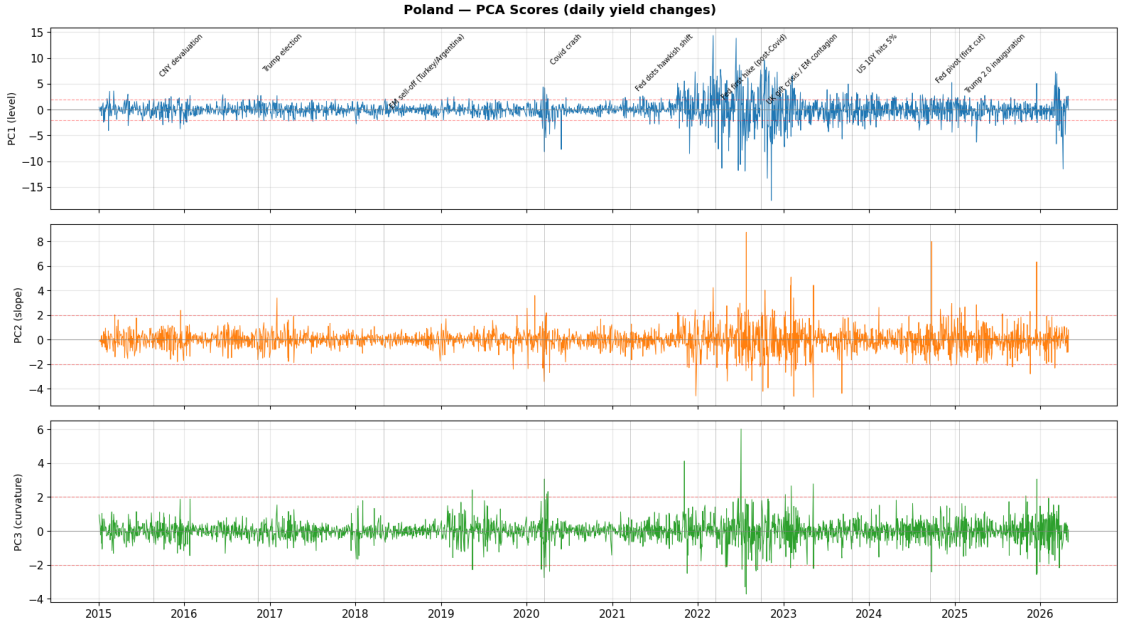
```
                    )

    axes[-1].xaxis.set_major_locator(mdates.YearLocator())
    axes[-1].xaxis.set_major_formatter(mdates.DateFormatter("%Y"))
    fig.suptitle(f"{country} - PCA Scores (daily yield changes)",␣
 ↪fontweight="bold")
    fig.tight_layout()
    plt.savefig(
        f"data/output/pca_scores_{country.lower().replace(' ', '_')}.png",
        dpi=150,
        bbox_inches="tight",
    )
    plt.show()


for country, res in sorted(pca_results.items()):
    plot_pc_scores(country, res["scores"], MACRO_EVENTS)
```

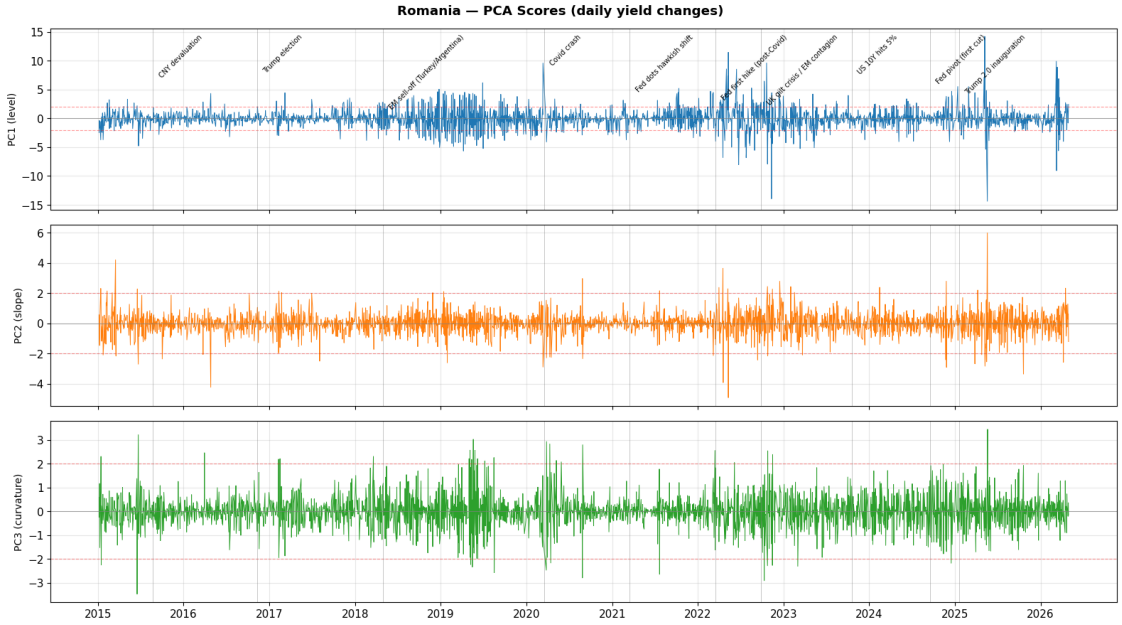


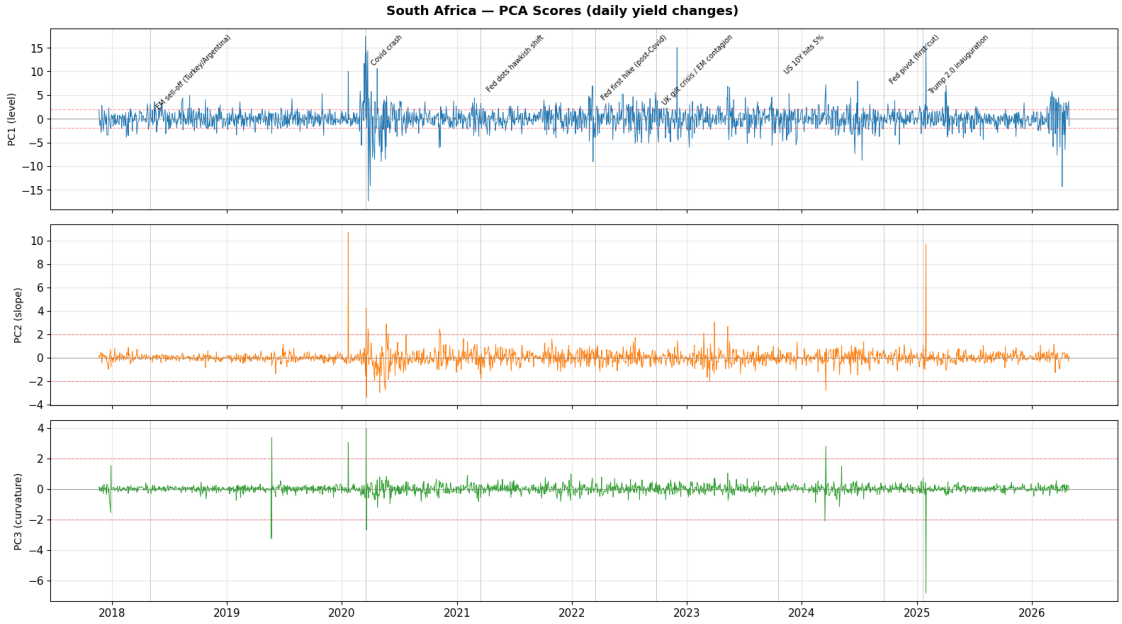

Colombia — PCA Scores (daily yield changes)
CNY devaluation
Trump election
EM sell-off (Turkey/Argentina)
Covid crash
Fed dots hawkish shift
Fed First hike (post-Covid)
UK gilt crisis / EM contagion
US 10Y hits 5%
Fed pivot (first cut)
Trump 2.0 inauguration
PC1 (level)
PC2 (slope)
PC3 (curvature)
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
2026


Hungary — PCA Scores (daily yield changes)
CNY devaluation
Trump election
EM sell-off (Turkey/Argentina)
Covid crash
Fed dots hawkish shift
Fed First hike (post-Covid)
UK gilt crisis / EM contagion
US 10Y hits 5%
Fed pivot (first cut)
Trump 2.0 inauguration
PC1 (level)
PC2 (slope)
PC3 (curvature)
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
2026


Mexico — PCA Scores (daily yield changes)
PC1 (level)
PC2 (slope)
PC3 (curvature)
CNY devaluation
Trump election
EM sell-off (Turkey/Argentina)
Covid crash
Fed dots hawkish shift
Fed pivot & hike (post-Covid)
Stagflation crises / EM contagion
US 10Y hits 5%
Fed pivot (first cut)
Trump 2.0 inauguration
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
2026


Poland — PCA Scores (daily yield changes)
PC1 (level)
PC2 (slope)
PC3 (curvature)
CNY devaluation
Trump election
EM sell-off (Turkey/Argentina)
Covid crash
Fed dots hawkish shift
Fed pivot & hike (post-Covid)
Stagflation crises / EM contagion
US 10Y hits 5%
Fed pivot (first cut)
Trump 2.0 inauguration
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
2026





[23]:
```
# Build a wide DataFrame: columns = "Country_MaturityY"
panel_parts = []
for country, dy in sorted(change_dfs.items()):
    renamed = dy.rename(columns={col: f"{country}_{col}" for col in dy.columns})
    panel_parts.append(renamed)
```

```
panel_dy = pd.concat(panel_parts, axis=1)

# Use the intersection of all date indices (inner join already handled by␣
 ↪concat)
# Drop any row with NaN (some countries have different trading calendars)
panel_dy = panel_dy.dropna()
print(f"Stacked panel: {panel_dy.shape[0]} obs × {panel_dy.shape[1]} series")

scaler_panel = StandardScaler()
panel_std = scaler_panel.fit_transform(panel_dy)

pca_panel = PCA(n_components=5)  # keep 5 to see explained variance drop-off
scores_panel = pca_panel.fit_transform(panel_std)

evr = pca_panel.explained_variance_ratio_
print("Panel PCA explained variance:")
for k in range(5):
    print(f"  PC{k+1}: {evr[k]:.1%}  (cumulative: {evr[:k+1].sum():.1%})")

# Panel scores DataFrame (first 3 for regime detection)
panel_scores_df = pd.DataFrame(
    scores_panel[:, :3],
    index=panel_dy.index,
    columns=["PC1 (global level)", "PC2 (global slope)", "PC3 (global␣
 ↪curvature)"],
)
```

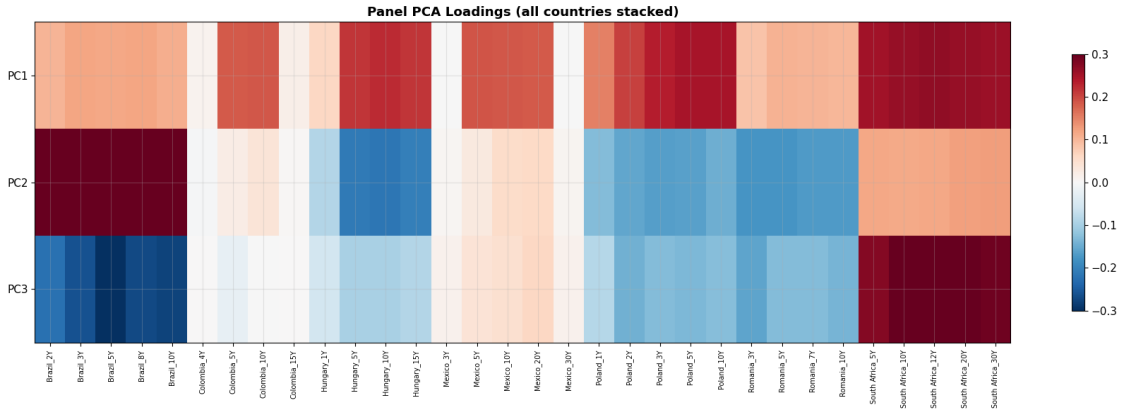
```
Stacked panel: 1346 obs × 32 series
Panel PCA explained variance:
  PC1: 22.1%  (cumulative: 22.1%)
  PC2: 11.9%  (cumulative: 34.0%)
  PC3: 10.0%  (cumulative: 43.9%)
  PC4: 7.7%  (cumulative: 51.6%)
  PC5: 7.1%  (cumulative: 58.7%)
```

[25]:
```
# Build regime features from per-country PC1 scores
pc1_panel = pd.DataFrame({
    country: res["scores"]["PC1 (level)"]
    for country, res in pca_results.items()
})
pc1_panel = pc1_panel.dropna()

regime_features = pd.DataFrame(index=pc1_panel.index)

# 1. Global level shock: average PC1 across countries
regime_features["avg_level"] = pc1_panel.mean(axis=1)
```

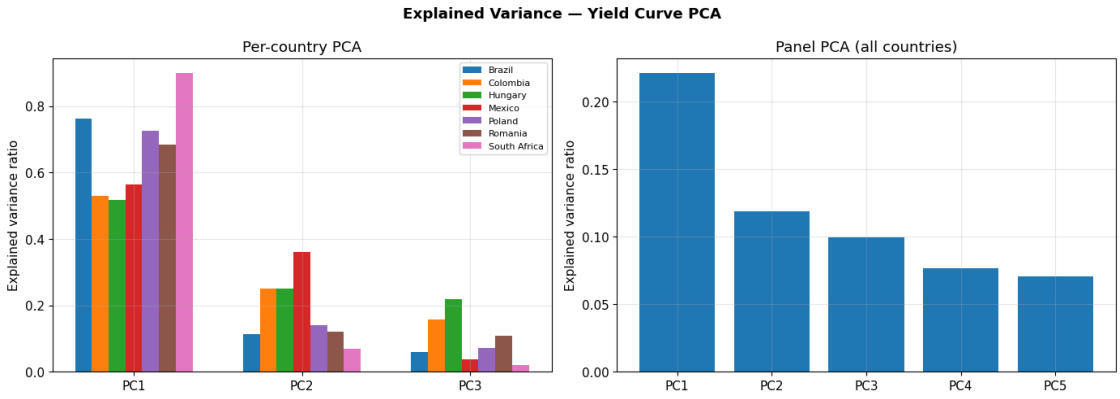
```
# 2. Cross-country dispersion: std of PC1 across countries
#    High = divergence/stress, Low = convergence/risk-on
regime_features["dispersion"] = pc1_panel.std(axis=1)

# 3. Realized vol: rolling 20-day std of the average level shock
regime_features["real_vol"] = regime_features["avg_level"].rolling(20).std()

regime_features = regime_features.dropna()
print(regime_features.describe().round(3))
```

```
       avg_level  dispersion  real_vol
count   1327.000    1327.000  1327.000
mean      -0.036       1.249     0.715
std        0.826       0.819     0.376
min       -5.332       0.198     0.215
25%       -0.418       0.733     0.438
50%       -0.043       1.025     0.602
75%        0.370       1.537     0.877
max        3.555       9.741     2.353
```

[26]:
```
fig, ax = plt.subplots(figsize=(18, 6))
loadings_panel = pd.DataFrame(
    pca_panel.components_[:3].T,
    index=panel_dy.columns,
    columns=["PC1", "PC2", "PC3"],
)
im = ax.imshow(loadings_panel.T.values, aspect="auto", cmap="RdBu_r", vmin=-0.
 ↪3, vmax=0.3)
ax.set_yticks(range(3))
ax.set_yticklabels(loadings_panel.columns)
ax.set_xticks(range(len(loadings_panel.index)))
ax.set_xticklabels(loadings_panel.index, rotation=90, fontsize=7)
plt.colorbar(im, ax=ax, shrink=0.8)
ax.set_title("Panel PCA Loadings (all countries stacked)", fontweight="bold")
fig.tight_layout()
plt.savefig("data/output/pca_panel_loadings.png", dpi=150, bbox_inches="tight")
plt.show()
```



[27]:
```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

# Left: per-country
countries_sorted = sorted(pca_results.keys())
x = np.arange(3)
width = 0.1
for i, country in enumerate(countries_sorted):
    evr_c = pca_results[country]["pca"].explained_variance_ratio_
    ax1.bar(x + i * width, evr_c, width, label=country)
ax1.set_xticks(x + width * 3)
ax1.set_xticklabels(["PC1", "PC2", "PC3"])
ax1.set_ylabel("Explained variance ratio")
ax1.set_title("Per-country PCA")
ax1.legend(fontsize=8)

# Right: panel
ax2.bar(range(5), pca_panel.explained_variance_ratio_, color="#1f77b4")
ax2.set_xticks(range(5))
ax2.set_xticklabels([f"PC{k+1}" for k in range(5)])
ax2.set_ylabel("Explained variance ratio")
ax2.set_title("Panel PCA (all countries)")

fig.suptitle("Explained Variance - Yield Curve PCA", fontweight="bold")
fig.tight_layout()
plt.savefig("data/output/pca_explained_variance.png", dpi=150,␣
 ↪bbox_inches="tight")
plt.show()
```


Explained Variance — Yield Curve PCA
Per-country PCA
Explained variance ratio
0.0
0.2
0.4
0.6
0.8
PC1
PC2
PC3
Brazil
Colombia
Hungary
Mexico
Poland
Romania
South Africa
Panel PCA (all countries)
Explained variance ratio
0.00
0.05
0.10
0.15
0.20
PC1
PC2
PC3
PC4
PC5